Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

- $Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)$
- $Waiting Time (WT) = Turnaround Time (TAT) - Burst Time (BT)$

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The next lines contain two integers for each process, where the first integer denotes the Arrival Time (AT) of the process and the second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

4

0 6

0 8
0 7

0 3

Sample Output 1

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00

Average Waiting Time = 7.00

Example 2

Sample Input 2

3

0 3

5 2

5 1

Sample Output 2

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Average Turnaround Time = 2.33

Average Waiting Time = 0.33

CODE:

```
#include <stdio.h> int
main()
{ int n;
   printf("Enter number of processes: ");
   scanf("%d", &n);

   int at[n], bt[n], tat[n], wt[n], done[n];

   // Input
   for(int i = 0; i < n; i++)
   { printf("Enter Arrival Time and Burst Time for P%d: ", i+1);
      scanf("%d %d", &at[i], &bt[i]);
      done[i] = 0;
   }

   int   completed   =   0;   int
   completionTime = 0; float
   avgtat = 0, avgwt = 0;

   while(completed < n)
   { int idx = -1; int
      minBT = 9999;

      // Find shortest job among arrived processes
      for(int i = 0; i < n; i++)
      { if(at[i] <= completionTime && done[i] == 0)
         { if(bt[i] < minBT)
            { minBT = bt[i];
               idx = i;
            }
         }
      }

      if(idx != -1)
      { if(completionTime < at[idx])
         {
            completionTime = at[idx];
         }

         completionTime += bt[idx];
```

```
            tat[idx] = completionTime - at[idx]; wt[idx]
            = tat[idx] - bt[idx];

            avgtat += tat[idx]; avgwt
            += wt[idx];

            done[idx] = 1;
            completed++;
        }
        else
        {
            completionTime++;
        }
    }

    printf("\nAverage Turnaround Time: %.2f\n", avgtat / n); printf("Average
    Waiting Time: %.2f\n", avgwt / n);
    return 0;
}
```

OUTPUT:

Custom Test Success

Input :

```
4
0 6
0 8
0 7
0 3
```

Expected Output :

```
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00
Average Waiting Time = 7.00
```

Output :

```
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00
Average Waiting Time = 7.00
```